MobilityDB Hiking Workshop

How to use MobilityDB to analyze hiking data

Alice Lombard and Raphaël Dubuget

COLLABORATORS

	TITLE : MobilityDB Hiking Workshop		
ACTION	*NAME*	*DATE*	*SIGNATURE*
WRITTEN BY	Alice Lombard and Raphaël Dubuget	July 9, 2024	

REVISION HISTORY

NUMBER	DATE	DESCRIPTION	NAME

Contents

1 Chapter 1: Treating the data **1**

1.1 Loading GPX data 1

2 Chapter 2: Visualizing the data **4**

2.1 Introduction 4

2.2 Connecting to the database 5

2.3 Visualizing the data 5

List of Figures

2.1 New PostGIS Connection 6
2.2 OpenStreetMap layer 7
2.3 Full temporal visualization 8
2.4 Visualizing the trip 9

List of Tables

2.1 Download the move.zip file 4

Abstract

This document is an example of how you can use MobilityDB in order to analyse data. It is directly inspired by the MobilityDB Workshop already existing. We will throughout this document present different use cases of MobilityDB, which is an extension on PostGIS and PostgreSQL.

This workshop was made by two students from INSA Rennes during an internship in June, July and August 2024.

We will use data from OpenStreetMap. The traces we are going to use in this example are rando_morbi1.gpx and rando_morbi2.gpx. GPX is a common file format for storing GPS data. It is used by many GPS devices and applications to store track logs and waypoints. In our case, a person did two hikes on the 26th of June 2024, in the Morbihan, and saved the data in two GPX files. Then, this person exported the data to OpenStreetMap and we can use his trace as we want. We will use these GPX files in this workshop. Please download the filesrando_morbi1.gpx and rando_morbi2.gpx from the OpenStreetMap website and put in a folder you will remember.

Now, we are ready to start the workshop. While this workshop illustrates the usage of MobilityDB functions, it does not explain them in detail. If you need help concerning the functions of MobilityDB, please refer to the documentation.

Chapter 1

Chapter 1: Treating the data

1.1 Loading GPX data

GPX, or GPS Exchange Format, is an XML data format for GPS data. Location data (and optionally elevation, time, and other information) is stored in tags and can be interchanged between GPS devices and software. Conceptually, a GPX file contains tracks, which are a record of where a moving object has been, and routes, which are suggestions about where it might go in the future. Furthermore, both tracks and routes and composed by points. The following is a truncated (for brevity) example GPX file.

```
<?xml version='1.0' encoding='UTF-8' standalone='yes' ?>
<gpx version="1.1"
xmlns="http://www.topografix.com/GPX/1/1"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.topografix.com/GPX/1/1
http://www.topografix.com/GPX/1/1/gpx.xsd"
creator="Example creator">
<metadata>
  <name>Dec 14, 2014 4:32:04 PM</name>
  <author>Example creator</author>
  <link href="https://..." />
  <time>2014-12-14T14:32:04.650Z</time>
</metadata>
<trk>
  <name>Dec 14, 2014 4:32:04 PM</name>
  <trkseg>
    <trkpt lat="30.16398" lon="31.467701">
      <ele>76</ele>
      <time>2014-12-14T14:32:10.339Z</time>
    </trkpt>
    <trkpt lat="30.16394" lon="31.467333">
      <ele>73</ele>
      <time>2014-12-14T14:32:16.00Z</time>
    </trkpt>
    <trkpt lat="30.16408" lon="31.467218">
      <ele>74</ele>
      <time>2014-12-14T14:32:19.00Z</time>
    </trkpt>
    [...]
  </trkseg>
  <trkseg>
    [...]
  </trkseg>
  [...]
</trk>
```

```
<trk>
  [...]
</trk>
[...]
<gpx>
```

To begin working on this file, you will need to transform the GPX data in order to use it with MobilityDB. The Python script below (called gpx_to_csv.py in our github) creates a CSV file based on the GPX one. It uses expat, a stream-oriented XML parser library.

```
import sys
import xml.parsers.expat

stack = []
def start_element(name, attrs):
stack.append(name)
if name == 'gpx' :
    print("lon,lat,time")
if name == 'trkpt' :
    print("{},{},".format(attrs['lon'], attrs['lat']), end="")

def end_element(name):
stack.pop()

def char_data(data):
if stack[-1] == "time" and stack[-2] == "trkpt" :
    print(data)

p = xml.parsers.expat.ParserCreate()

p.StartElementHandler = start_element
p.EndElementHandler = end_element
p.CharacterDataHandler = char_data

p.ParseFile(sys.stdin.buffer)
```

To use it, you can copy and paste this code into a Python file and run it with the following command (make sure to indicate the paths for each file):

```
python3 python_file.py < example.gpx > example.csv
```

You can have different names for the gpx and csv files if you want, but make sure to change the names in the command.

This is what the resulting CSV file will look like, with one column for the longitudes, one for the latitudes and the last one for the timestamps:

```
lon,lat,time
-2.869371,47.551691,2024-06-26T14:04:15+02:00
-2.86959,47.551946,2024-06-26T14:04:40+02:00
-2.86982,47.55215,2024-06-26T14:05:04+02:00
...
```

This CSV file can then be loaded into MobilityDB as follows (be carefull to read the comments as you cannot just run everything together):

```
DROP EXTENSION IF EXISTS mobilitydb CASCADE;
CREATE EXTENSION mobilityDB CASCADE;

DROP TABLE IF EXISTS trips_input;
CREATE TABLE trips_input (
    id int,
    lon float,
```

```
    lat float,
    time timestamptz
);

INSERT INTO trips_input VALUES
(0,NULL,NULL,NULL);


-- Here you need to import each trip file separatly.
-- This allows you to put a different ID on each trip, because if you import all of them at ↩
     the same time it becomes difficult to tell apart the trips from one another.

-- First run the following SQL command (you need to change the name of the file you want to ↩
     import and the path to it)
COPY trips_input(lon, lat, time) FROM
'/home/gpx_data/example.csv' DELIMITER ',' CSV HEADER;

-- Do the above or run the command below in a terminal if you have a permission error
-- (you need to change the name of the database, and the name and path to the file you want ↩
     to import)
-- psql -d DataBaseName -c "\copy trips_input(lon, lat, time) FROM '/home/gpx_data/exemple. ↩
    csv' DELIMITER ',' CSV HEADER;"

-- Then run this command to update the ID
WITH max_id_cte AS (
    SELECT MAX(id)+1 AS max_id FROM trips_input
),
rows_to_update AS (
    SELECT id, (SELECT max_id FROM max_id_cte) AS new_id
    FROM trips_input
    WHERE id IS NULL
)
UPDATE trips_input SET id = rows_to_update.new_id
FROM rows_to_update
WHERE trips_input.id IS NULL;

-- Now you can repeat the last two steps for each trip file you want to import. Don’t  ↩
    forget to change the name of the file each time!


DROP TABLE IF EXISTS trips_mdb;
CREATE TABLE trips_mdb (
    id int,
    date date,
    trip tgeompoint,
    trajectory geometry,
    PRIMARY KEY (id)
);

INSERT INTO trips_mdb(id, date, trip)
SELECT id, date(time), tgeompointSeq(array_agg(tgeompoint(
    ST_SetSRID(ST_Point(lon, lat), 4326), time) ORDER BY time))
FROM trips_input
WHERE id != 0
GROUP BY id, date;

UPDATE trips_mdb
SET trajectory = trajectory(trip);
```

Chapter 2

Chapter 2: Visualizing the data

2.1 Introduction

Now that we have cleaned the data and made it ready to be used, we will visualize the data in order to have a better understanding of it. To do so, we are going to use QGIS, a free and open-source geographic information system. You can download it here. If you're interested, know that we are going to follow this tutorial: MobilityDB: Hands on Tutorial on Managing and Visualizing Geospatial Trajectories in SQL. We will use the plugin Move to visualize the data. Here is the repo where you can find it. Please click on the move.zip file, and download it. Take care to do it like in this following image, otherwise it could not be downloaded correctly. Also, put it in a folder you will remember. Now, open QGIS and click on Plugins → Manage and Install Plugins. Click

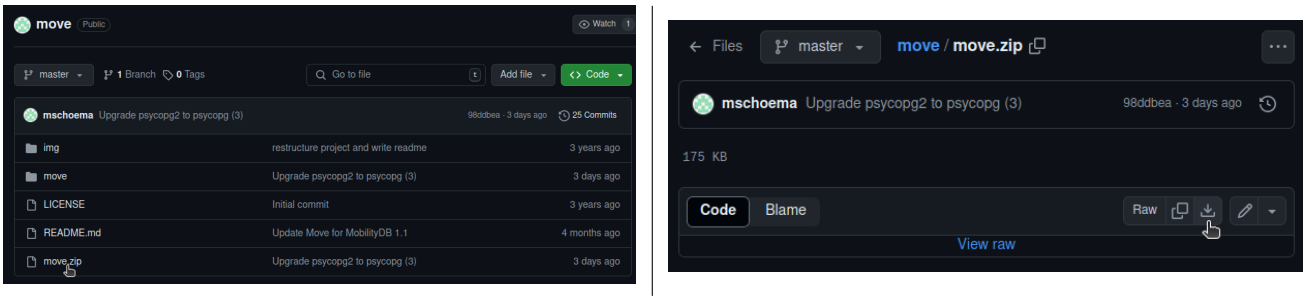

Table 2.1: Download the move.zip file

on the install from zip button and select the move.zip file you just downloaded. Then, you can use the plugin to visualize the data. A MobilityDB logo will appear on the up-left corner of the screen.

You may have an error saying:

```
Couldn't load plugin 'move' due to an error when calling its classFactory() method
    ModuleNotFoundError: No module named 'psycopg'
```

To fix this, you need to install the psycopg module. You can do it by running the following command in a terminal:

```
pip install psycopg
```

If you've had to install the psycopg module, you will need to uninstall the incorrect move plugin that you just installed (Plugins → Manage and Install Plugins → Installed → Move → Uninstall Plugin) and then reinstall it. You may also need to restart QGIS to make the plugin work.

2.2 Connecting to the database

You can now create a new blank project in QGIS. Then in the browser on the left of the screen, you should find a 'PostGIS' line. Right click on it and then 'New Connection...'. You will see this window:

Here is what you need to fill out:

- Name: choose a name for this connection
- Host: if the database is hosted on your machine, then 'localhost'. If not, you should be able to fgure it out by yourself.
- Port: again, if self hosted then keep '5432', if not you should know.
- Database: here write the name of the database in which you have loaded the data.
- Authentication: go to the 'Basic' tab and fill out the user name and password for the account that has access to the database. Make sure to check the 'Store' boxes next to User name and Password.

You can now click OK and find your connection, database and the trips_mdb table in the tree below the PostGIS line in the QGIS Browser.

2.3 Visualizing the data

Now that you have connected to the database, you can visualize the data. First, let's put a map layer. In the browser window (on the left of the screen), click on 'XYZ Tiles' and then right click on 'OpenStreetMap' and select 'Add Layer to Project'. You will see an OpenStreetMap layer in the inferior left part of the screen. You can zoom in the part you are studyin. In our case, we are focusing on a part of France, in Britanny, called Morbihan. Then, let's add the data we have loaded in the database. First, let's draw the trajectory. Click on PostGIS → connection_name → public → trips_mdb → Fields → trajectory. You will see the trajectory in the map. Now, we want to visualize the moving point of the trip. To do so, you need to click on the 'Move' logo on the up-left side of the screen. Select your database and the table you want to visualize. Then, you can write a query and execute it. We are going to put in this query:

```
SELECT date, trip FROM trips_mdb;
```

Now click on 'Execute Query'. Let's see the trip moving while we are visualizing it. To do so, click on View → Panels → Temporal Controller. You will see a window like this: Now this is your time to play with the data and visualize it as you want. You can put the time in the temporal controller and see the trip moving. You can also change the color of the trip, the size of the points, etc.

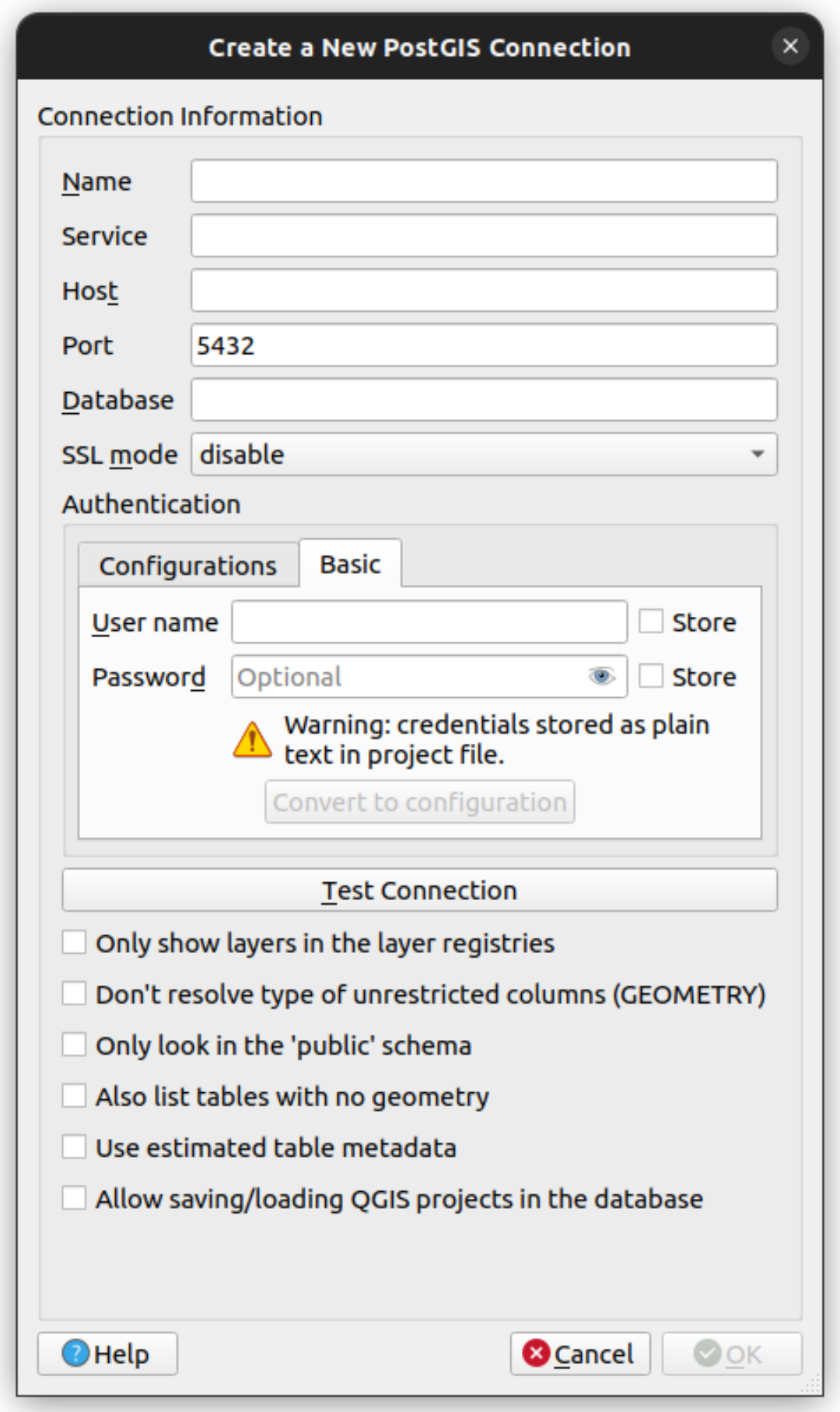

Figure 2.1: New PostGIS Connection

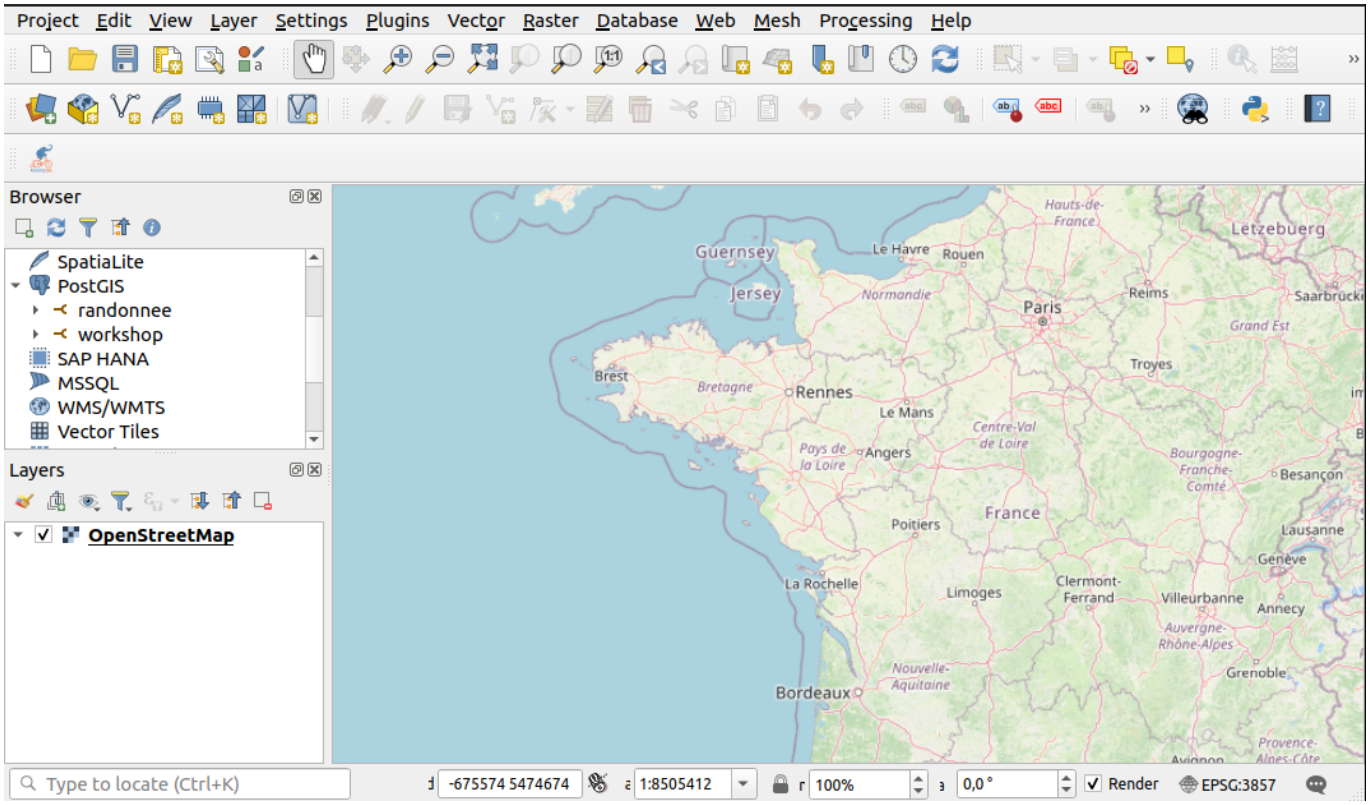

Figure 2.2: OpenStreetMap layer

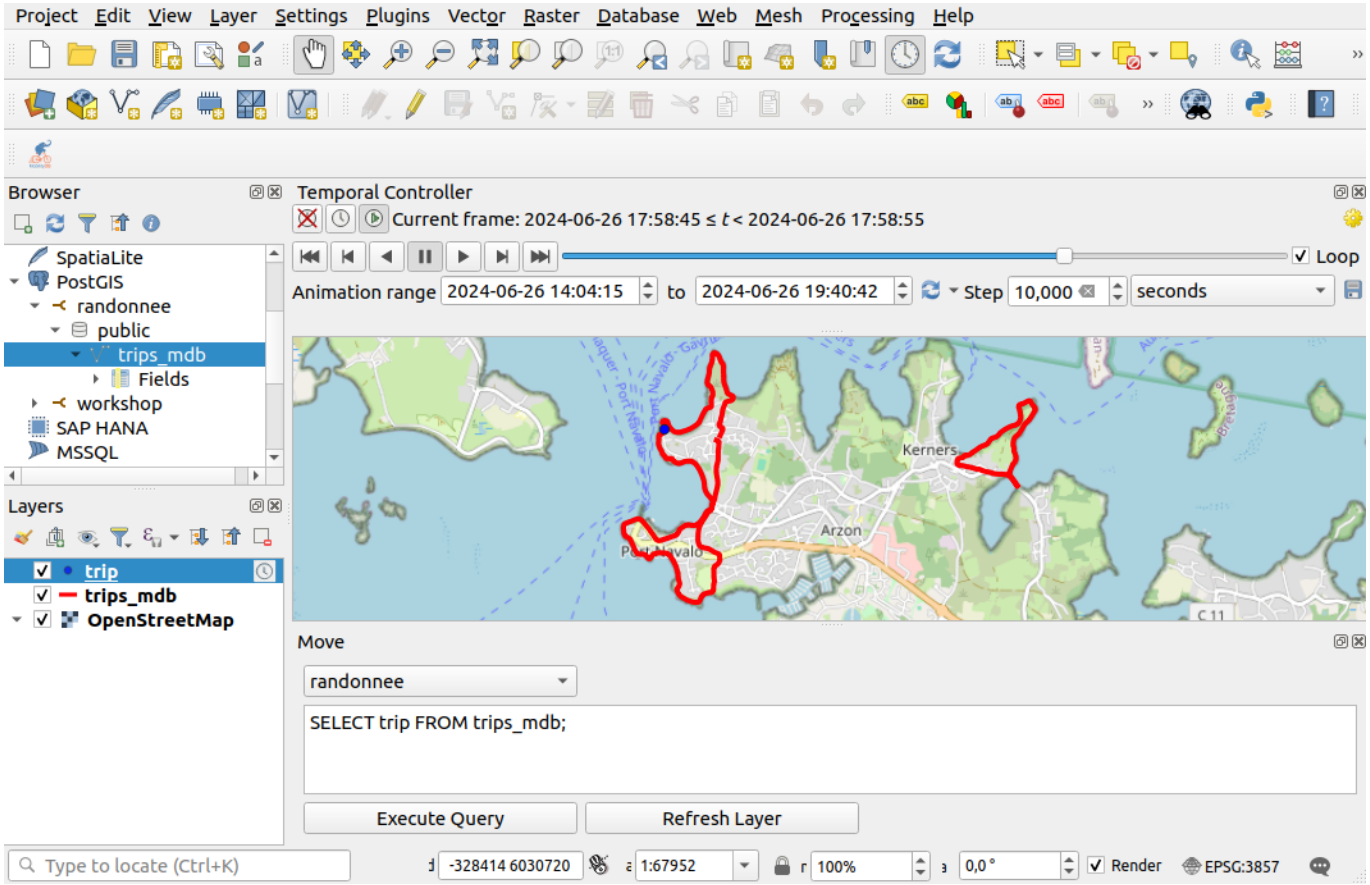

Figure 2.3: Full temporal visualization

Figure 2.4: Visualizing the trip